

WIPRO RGA PROGRAM 2025 — CAPSTONE PROJECT

NexCare Project Report

Interactive Dashboard System for Customer Service Operations
Complete Technical Documentation — Every File, Every Function

Presented by: Rishitha Manyam | Java Full Stack | Spring Boot 3 + Angular 17 | June 2026

00

Table of Contents

1. What is NexCare — Project Overview	Section 01
2. How It All Works — The Big Picture	Section 02
3. Project Folder Structure	Section 03
4. Backend — Spring Boot (Every File Explained)	Section 04
5. Frontend — Angular 17 (Every File Explained)	Section 05
6. Database Tables & Columns	Section 06
7. API Endpoints Reference	Section 07
8. Key Flows — Step by Step	Section 08
9. Security Architecture	Section 09
10. Configuration Files	Section 10

01

What is NexCare — Project Overview

NexCare is a **full-stack web application** built for telecom companies in India to manage their customer service operations. The application provides a **single unified platform** for four different types of users — Admin, Manager, Service Representative, and Customer — each with their own dedicated dashboard and strictly controlled access.

Before NexCare, customer service teams would use separate tools: spreadsheets for tickets, email for customer communication, and manual reports for performance. NexCare replaces all of this with one real-time, secure, role-based system.

What Each Role Can Do

Role	What they can do
Admin	Add/edit/delete employees, see all tickets, report service outages, view company-wide analytics with charts
Manager	Monitor team ticket performance, view rep metrics, manage outages, see team-level analytics
Representative	Pick up customer tickets, update ticket status, search customers, track their own performance stats
Customer	Raise support tickets, track ticket status, rate resolved tickets, check for outages in their city, use the chatbot

Technology Used

Part	Technology	Purpose
Frontend	Angular 17	The website the user sees and interacts with
Backend	Spring Boot 3.2.5 (Java)	The server that processes requests and talks to the database
Database	H2 File Database	Stores all data — users, tickets, outages, notifications

Security	JWT + Spring Security	Ensures only logged-in users with the right role can access data
Charts	Chart.js	Bar and doughnut charts on the dashboards
Build Tool	Maven	Downloads Java dependencies and builds the backend
Package Manager	npm	Downloads JavaScript/Angular dependencies for the frontend

02 How It All Works — The Big Picture

Think of NexCare as two separate programs running at the same time and talking to each other:

Frontend (Angular) runs in the user's browser on port 4200. It shows the UI.

Backend (Spring Boot) runs as a Java server on port 8080. It handles data.

When Angular needs data, it sends an HTTP request to Spring Boot, which reads/writes the database and sends back a JSON response.

The Complete Request Journey

1

User opens browser → Angular loads

The browser loads `index.html` from the Angular app. Angular's router reads the URL and decides which page component to show. If the URL is `/#/login`, it loads the Login page.

2

User enters email + password → clicks Sign In

The login form collects credentials and calls `AuthService.login()` which sends a `POST` `/api/auth/login` request to the Spring Boot server (via the proxy on port 8080).

3

Spring Boot validates credentials

`AuthController` receives the request, calls `AuthService`, which uses `BCryptPasswordEncoder` to verify the hashed password stored in the H2 database. If correct, `JwtUtil` generates a signed token containing the user's email, role, and `userId`.

4

Token returned to Angular → stored in sessionStorage

The JSON response contains `{ token: "eyJ...", user: { name, email, role } }`. Angular stores both in `sessionStorage` (wiped when the browser tab closes).

5

Angular redirects to the correct dashboard by role

The login component reads `user.role` and calls `router.navigate(['/admin'])` or `['/customer']` etc. The Auth Guard checks the token and role before allowing the page to load.

6

Dashboard loads → fetches data from backend

Every API call automatically gets the JWT token attached to its headers by the JWT Interceptor. Spring Boot's `JwtAuthFilter` validates this token on every request before allowing any data to be returned.

7

User interacts → Angular updates UI, Spring Boot updates database

Whether raising a ticket, updating a status, or reporting an outage — Angular sends the right API call, Spring Boot processes it, saves to H2, and returns the updated data. Angular re-renders the UI automatically.

03 Project Folder Structure

```

NexCare/
├── src/                                ← Spring Boot backend (Java)
│   ├── main/
│   │   ├── java/com/wipro/csd/
│   │   │   ├── CsdApplication.java    ← Main entry point
│   │   │   ├── config/                ← Security, JWT, seed data
│   │   │   ├── controller/            ← REST API endpoints
│   │   │   ├── dto/                  ← Data Transfer Objects
│   │   │   ├── filter/                ← JWT request filter
│   │   │   ├── model/                 ← Database entities
│   │   │   ├── repository/            ← Database query interfaces
│   │   │   └── service/               ← Business logic
│   │   └── resources/
│   │       └── application.properties ← All app settings
│   └── frontend/                      ← Angular frontend (TypeScript)
│       ├── src/
│       │   ├── main.ts                ← Angular entry point
│       │   ├── index.html             ← Single HTML page
│       │   ├── styles.css              ← Global styles & theme
│       │   └── app/
│       │       ├── app.config.ts       ← App-wide setup
│       │       ├── app.routes.ts       ← URL routing
│       │       ├── guards/             ← Route protection
│       │       ├── interceptors/       ← Adds token to HTTP calls
│       │       ├── models/             ← TypeScript interfaces
│       │       ├── services/           ← API call functions
│       │       ├── pages/              ← Individual page components
│       │       └── shared/              ← Reusable components
│       ├── proxy.conf.json             ← Forwards /api to port 8080
│       ├── angular.json                ← Angular build config
│       └── package.json                ← npm dependencies
├── docs/                              ← All documentation
├── .vscode/                           ← VS Code run tasks
├── pom.xml                             ← Maven dependencies
└── .gitignore

```

04

Backend — Spring Boot (Every File Explained)

Entry Point

ENTRY POINT

CsdApplication.java

WHAT IT DOES

This is the first Java file that runs when you start the application. It contains the `public static void main(String[] args)` method — Java always starts from here. The `@SpringBootApplication` annotation tells Spring to scan all classes in the project and wire everything together automatically.

THE CODE

```
@SpringBootApplication
public class CsdApplication {
    public static void main(String[] args) {
        SpringApplication.run(CsdApplication.class, args);
    }
}
```

WHY IT MATTERS

Without this file, nothing runs. It's the ignition key of the entire Spring Boot application.

Configuration Files

CONFIG

src/main/resources/application.properties

WHAT IT DOES

All application settings in one file. Spring Boot reads this automatically on startup. No code changes needed to switch the port, database, or token expiry — just change this file.

KEY SETTINGS AND WHAT THEY MEAN

Setting	Value	Meaning
<code>server.port</code>	8080	Backend runs on port 8080

<code>spring.datasource.url</code>	<code>jdbc:h2:file:./data/csddb</code>	H2 database stored as a file named csddb in the /data folder
<code>spring.jpa.hibernate.ddl-auto</code>	<code>update</code>	Automatically creates/updates database tables to match the Java model classes
<code>spring.h2.console.enabled</code>	<code>true</code>	Enables a browser-based database viewer at /h2-console
<code>jwt.secret</code>	WiproCapstone...	The secret key used to sign JWT tokens
<code>jwt.expiration</code>	86400000	Tokens expire after 86400000ms = 24 hours

RESILIENCE4J SETTINGS

The file also configures a Circuit Breaker for the ticket service. If the ticket service fails 50% of the time over 10 requests, the circuit "opens" and returns an empty list instead of crashing — preventing cascading failures.

SECURITY `config/SecurityConfig.java`

WHAT IT DOES

Configures the entire security layer of the application. It tells Spring Security which URLs are open to everyone (login, register) and which require authentication (everything else). It also registers the JWT filter so every request gets its token validated.

HOW IT WORKS

Public endpoints: `POST /api/auth/login` and `POST /api/auth/register` — no token needed.

All other endpoints: Must have a valid JWT token. If not, Spring returns 401 Unauthorized automatically.

CORS: Configured to allow the Angular app on port 4200 to call the backend on port 8080.

Password encoder: Registers BCryptPasswordEncoder as a Spring Bean so it can be injected anywhere in the app.

JWT `config/JwtUtil.java`

WHAT IT DOES

The only class that creates and reads JWT tokens. Think of a JWT token like a signed membership card — JwtUtil is the machine that prints and verifies these cards.

METHODS

Method	What it does
<code>generateToken(user)</code>	Creates a JWT token containing the user's email, userId, role, name, and managerId. Signs it with the secret key. Sets expiry to 24 hours.
<code>extractEmail(token)</code>	Reads the email stored inside a token
<code>extractRole(token)</code>	Reads the role (ADMIN/MANAGER/REPRESENTATIVE/CUSTOMER) from the token
<code>extractUserId(token)</code>	Reads the userId number from the token
<code>isTokenValid(token, email)</code>	Checks: (1) email in token matches the provided email, (2) token is not expired. Returns true only if both pass.

SEED DATA `config/DataInitializer.java`

WHAT IT DOES

Automatically creates demo data when the application starts for the first time. Uses the `@PostConstruct` annotation which means Spring calls the `init()` method automatically right after startup.

WHAT IT CREATES

Type	Count	Details
Admin	1	admin@nexcare.in / admin123
Managers	2	manager1 and manager2 @nexcare.in
Representatives	4	amit.kumar, ravi.singh, deepa.nair, preethi.iyer @nexcare.in
Customers	4	priya.sharma, rahul.verma, ananya.krishnan, vikram.patel
Tickets	10	Mix of OPEN, IN_PROGRESS, CLOSED across all domains
Outages	2	Mumbai (Andheri, Bandra) and Delhi (Connaught Place, Saket)
Notifications	4	Sample outage notification for each rep

SMART CHECK

Before creating anything, it checks if an admin user already exists: `if (userRepo.existsByEmail("admin@nexcare.in")) return;` — so data is only seeded once, never duplicated on restart.

Security Filter

FILTER filter/JwtAuthFilter.java

WHAT IT DOES

This class runs for *every single HTTP request* that comes into the backend. It extends `OncePerRequestFilter` which guarantees it runs exactly once per request. It reads the `Authorization: Bearer <token>` header, validates the token, and if valid, tells Spring Security who the user is.

STEP BY STEP

1. Read the `Authorization` header from the incoming request.
2. If the header is missing or doesn't start with "Bearer ", skip validation and continue (Spring Security will block it later if the route is protected).
3. Extract the email from the token using `JwtUtil`.
4. Load the user from the database using that email.
5. Check if the token is valid for that user.
6. If valid, create a `UsernamePasswordAuthenticationToken` and put it in Spring's `SecurityContextHolder` — this is how Spring knows the request is authenticated.
7. Continue to the actual controller.

Models (Database Entities)

Models are Java classes that map directly to database tables. Each field becomes a column. The `@Entity` annotation tells Spring to create/update the corresponding table automatically.

MODEL model/User.java

MAPS TO THE USERS TABLE IN THE DATABASE

Field	Type	Description
id	Long	Auto-generated unique number for each user
name	String	Full name
email	String (unique)	Used as the login username
password	String	Stored as a BCrypt hash — never plain text
role	Enum	ADMIN, MANAGER, REPRESENTATIVE, or CUSTOMER
phone	String	Indian format (9876543XXX)

location	String	City (Mumbai, Delhi, Bangalore...)
managerId	Long	For reps: links them to their manager
firstLogin	boolean	True if rep hasn't changed their initial password
createdAt	LocalDateTime	When the account was created

The `@Lombok @Data` annotation auto-generates getters, setters, equals, and toString. `@Builder` allows clean object creation like `User.builder().name("Rishitha").role(CUSTOMER).build()`.

MODEL `model/Ticket.java`

MAPS TO THE TICKETS TABLE — THE CORE BUSINESS ENTITY

Field	Type	Description
id	Long	Ticket number (e.g., #1, #2, #3)
customerId / customerName	Long / String	Who raised the ticket
customerPhone / customerLocation	String	Customer contact details (stored on ticket for history)
domain	String	Internet Services, Mobile, DTH, etc.
subject / description	String	The ticket title and full details
status	Enum	OPEN, IN_PROGRESS, or CLOSED
repId / repName	Long / String	The rep who picked it up (null if unassigned)
managerId	Long	Set when a rep claims the ticket — links to their manager
createdAt	LocalDateTime	When the ticket was raised
firstResponseAt	LocalDateTime	When a rep first set it to IN_PROGRESS
resolvedAt	LocalDateTime	When the ticket was CLOSED
responseTimeHrs	Double	Hours from creation to first response
resolutionTimeHrs	Double	Hours from creation to close

rating	Integer	1–5 star rating from the customer (only on CLOSED tickets)
--------	---------	--

MODEL `model/Outage.java`

MAPS TO THE OUTAGES TABLE

Fields: `id`, `location` (city), `affectedAreas` (e.g., "Andheri, Bandra, Colaba"), `domain` (e.g., Internet Services), `severity` (HIGH/MEDIUM/LOW), `description`, `status` (ACTIVE/RESOLVED), `reportedAt`, `resolvedAt`. Customers see an alert banner only when an ACTIVE outage matches their city.

MODEL `model/Notification.java`

MAPS TO THE NOTIFICATIONS TABLE

Fields: `id`, `userId`, `message`, `type` (TICKET/STATUS/OUTAGE), `isRead` (boolean), `createdAt`. Created automatically when a new ticket is raised (notifies reps) or when a ticket status changes (notifies the customer).

Repositories (Database Queries)

Repositories are interfaces that extend `JpaRepository`. Spring Data JPA automatically generates all SQL queries — you just write a method name and Spring figures out the SQL.

REPO `repository/TicketRepository.java`

CONTAINS ALL TICKET DATABASE QUERIES

Method name	SQL it generates
<code>findByCustomerIdOrderByCreatedAtDesc</code>	SELECT * FROM tickets WHERE customer_id=? ORDER BY created_at DESC
<code>findByRepIdOrderByCreatedAtDesc</code>	SELECT * FROM tickets WHERE rep_id=? ORDER BY created_at DESC
<code>countByStatus</code>	SELECT COUNT(*) FROM tickets WHERE status=?
<code>averageRating</code>	SELECT AVG(rating) FROM tickets WHERE rating IS NOT NULL

countByDomain

SELECT domain, COUNT(*) FROM tickets GROUP BY domain

searchByCustomer

Custom @Query: searches by customer name, phone, or location (LIKE %query%)

REPO repository/UserRepository.java

CONTAINS ALL USER DATABASE QUERIES

Key methods: `findByEmail(email)` — used by login, JWT filter, and auth guard. `findByRole(role)` — used to list all reps when notifying of new tickets. `findByManagerId(managerId)` — used by managers to see their team. `countByRole(role)` — used by analytics. `existsByEmail(email)` — used by DataInitializer to check if seed data already exists.

DTOs (Data Transfer Objects)

DTOs are simple classes used to receive data from the frontend or send data back. They prevent exposing internal model fields directly over the API.

DTO dto/ (all DTO files)

File	Used for	Fields
LoginRequest.java	Receives login form data	email, password
LoginResponse.java	Sends token + user back to Angular	token, user object
RegisterRequest.java	Receives Sign Up form data	name, email, phone, location, password
TicketRequest.java	Receives new ticket data from customer	domain, subject, description
StatusUpdateRequest.java	Receives ticket status change from rep	status (OPEN/IN_PROGRESS/CLOSED)
UserRequest.java	Receives profile update data	name, phone, location

ChangePasswordRequest.java	Receives password change form	currentPassword, newPassword
ApiResponse.java	Simple success/error message wrapper	success (boolean), message

Services (Business Logic)

Services contain the actual business logic. Controllers call services, services call repositories. This separation keeps the code clean and testable.

SERVICE service/AuthService.java

HANDLES ALL AUTHENTICATION OPERATIONS

login(): Finds user by email, uses BCryptPasswordEncoder to check if the submitted password matches the stored hash. If match, calls JwtUtil.generateToken() and returns LoginResponse with the token and user object.

register(): Checks if email already exists, hashes the password with BCrypt, creates a new CUSTOMER user, saves to database.

changePassword(): Verifies current password, hashes new password, saves.

updateProfile(): Updates name, phone, location for the logged-in user.

SERVICE service/TicketService.java

HANDLES ALL TICKET OPERATIONS

getTicketsForUser(): Uses a switch on the user's role to return the right tickets. Customers see only their own tickets. Reps see tickets assigned to them. Managers see their team's tickets. Admins see all. This method has a `@CircuitBreaker` annotation — if it fails repeatedly, the fallback returns an empty list instead of crashing.

createTicket(): Builds a new Ticket object from the customer's data and the request form, saves it, then sends a notification to all reps that a new ticket has been raised.

updateStatus(): When a rep sets a ticket to IN_PROGRESS, it records their name, the firstResponseAt time, and calculates responseTimeHrs. When set to CLOSED, it records resolvedAt and calculates resolutionTimeHrs. Notifies the customer of the status change.

rateTicket(): Verifies the customer owns the ticket, then saves the 1–5 star rating.

SERVICE service/AnalyticsService.java

CALCULATES ALL DASHBOARD STATISTICS

getAdminStats(): Returns total counts of customers, managers, reps, tickets by status, average satisfaction rating, and a map of tickets grouped by domain and location for the charts.

getManagerStats(): Returns their team size, team ticket counts, and for each rep: their open/in-progress/closed ticket count plus average response and resolution times.

getRepStats(): Returns this rep's own ticket counts and average response/resolution times.

getCustomerStats(): Returns this customer's own ticket counts by status.

SERVICE `service/UserDetailsServiceImpl.java`**REQUIRED BY SPRING SECURITY**

Implements Spring's `UserDetailsService` interface. Spring Security calls

`loadUserByUsername(email)` during JWT validation to load the user from the database. Returns a `UserDetails` object that Spring uses internally to know what roles the user has.

Controllers (REST API Endpoints)

Controllers receive HTTP requests from Angular, call the appropriate service, and return a JSON response. Each controller class handles a specific category of functionality.

CONTROLLER `controller/AuthController.java`

Method	Endpoint	Auth Required	What happens
POST	/api/auth/login	No	Validates credentials, returns JWT token + user
POST	/api/auth/register	No	Creates new CUSTOMER account
GET	/api/auth/me	Yes	Returns current logged-in user's profile
PUT	/api/auth/change-password	Yes	Changes user's password
PUT	/api/auth/profile	Yes	Updates name, phone, location

CONTROLLER `controller/TicketController.java`

Method	Endpoint	Roles	What happens
GET	/api/tickets	All roles	Returns tickets filtered by the caller's role
POST	/api/tickets	CUSTOMER only	Creates a new support ticket
PUT	/api/tickets/{id}/status	REP, MANAGER, ADMIN	Updates ticket status, records timing
PUT	/api/tickets/{id}/rate	CUSTOMER only	Submits a 1–5 star rating for a closed ticket
GET	/api/tickets/search?q=	REP, MANAGER, ADMIN	Searches tickets by customer name/phone/location

The `@PreAuthorize` annotation on each method enforces role-based access at the method level — even if someone bypasses the frontend, the backend rejects unauthorised calls.

CONTROLLER `controller/AnalyticsController.java`

Provides `GET /api/analytics/admin`, `/manager`, `/rep`, `/customer`. Calls the matching `AnalyticsService` method and returns a Map of statistics. The Angular dashboard cards and Chart.js charts all read from these endpoints.

CONTROLLER `controller/OutageController.java`

`GET /api/outages` — Returns all outages (for admin/manager). `POST /api/outages` — Admin reports a new outage. `PUT /api/outages/{id}/resolve` — Marks an outage as RESOLVED. `GET /api/outages/location?city=Mumbai` — Returns ACTIVE outages for a specific city (used by Customer dashboard to show the alert banner).

CONTROLLER `controller/NotificationController.java`

`GET /api/notifications` — Returns unread notifications for the logged-in user. `PUT /api/notifications/{id}/read` — Marks a notification as read. Notifications are shown in the top bar bell icon across all dashboards.

05 Frontend — Angular 17 (Every File Explained)

Entry Points

ENTRY frontend/src/main.ts

The first TypeScript file Angular runs. It calls `bootstrapApplication(AppComponent, appConfig)` — this starts Angular with the root component and the application configuration. One line is the entire file. Angular handles everything else.

ENTRY frontend/src/index.html

The single HTML page for the entire application. Contains only one custom tag: `<app-root></app-root>`. Angular replaces this with all the page content dynamically. No page reloads happen — Angular is a Single Page Application (SPA).

Application Configuration

CONFIG app/app.config.ts

BOOTSTRAPS THE ANGULAR APPLICATION WITH ALL PROVIDERS

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withHashLocation()),
    provideHttpClient(withInterceptors([jwtInterceptor])),
  ]
};
```

provideRouter with withHashLocation(): Sets up routing and makes URLs use the `#` character (e.g., `/#/admin?tab=dashboard`). Hash URLs are required for GitHub Pages hosting — without the hash, refreshing the page would return a 404.

provideHttpClient with withInterceptors([jwtInterceptor]): Sets up the HTTP client and registers the JWT interceptor globally. Every HTTP call in the entire app now automatically includes the auth token.

ROUTING `app/app.routes.ts`**DEFINES ALL PAGE URLS AND WHO CAN ACCESS THEM**

URL Path	Component loaded	Required role
/login	LoginComponent	None (public)
/admin	AdminComponent	ADMIN
/manager	ManagerComponent	MANAGER
/representative	RepresentativeComponent	REPRESENTATIVE
/customer	CustomerComponent	CUSTOMER
/profile	ProfileComponent	Any authenticated user
/** (anything else)	Redirects to /login	—

Each protected route has `canActivate: [authGuard]` and `data: { role: 'ADMIN' }`. Every component is lazy-loaded using `loadComponent: () => import('./pages/...')` — the code for each page is only downloaded when that user actually navigates to it, making the initial load much faster.

Security (Frontend)

GUARD `app/guards/auth.guard.ts`**PROTECTS ROUTES — RUNS BEFORE ANY PAGE LOADS**

```
export const authGuard: CanActivateFn = (route) => {
  const token = sessionStorage.getItem('token');
  const user = JSON.parse(sessionStorage.getItem('user') || 'null');
  if (!token || !user) { router.navigate(['/login']); return false; }
  const requiredRole = route.data?.['role'];
  if (requiredRole && user.role !== requiredRole) { router.navigate(['/login']); return false; }
  return true;
};
```

Two checks: (1) Is there a token in sessionStorage? (2) Does the user's role match the required role for this route? Both must pass or the user is sent back to login. This runs on *every* navigation — not just the first load.

INTERCEPTOR `app/interceptors/jwt.interceptor.ts`**AUTOMATICALLY ADDS THE JWT TOKEN TO EVERY HTTP REQUEST**

```
export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
  const token = sessionStorage.getItem('token');
  if (token) {
    req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  }
  return next(req);
};
```

This is a functional interceptor (Angular 17 style — no class needed). Every time Angular makes an HTTP call, this runs first. If a token exists, it clones the request and adds the Authorization header. The original request is immutable — that's why it uses `req.clone()`.

TypeScript Models

MODELS `app/models/ (all 4 model files)`

TypeScript interfaces that mirror the Java model classes. They give type safety — TypeScript will warn you at compile time if you try to access a field that doesn't exist.

File	Fields
<code>user.model.ts</code>	id, name, email, role, phone, location, managerId, firstLogin, createdAt
<code>ticket.model.ts</code>	id, customerId, customerName, domain, subject, status, repId, repName, createdAt, responseTimeHrs, resolutionTimeHrs, rating
<code>outage.model.ts</code>	id, location, affectedAreas, domain, severity, status, description, reportedAt, resolvedAt
<code>notification.model.ts</code>	id, userId, message, type, isRead, createdAt

Angular Services

SERVICE `app/services/auth.service.ts`**FRONTEND AUTHENTICATION — ALL LOGIN/LOGOUT/PROFILE LOGIC**

Key methods: `login()` — calls POST `/api/auth/login`, saves token + user to `sessionStorage`, returns user. `register()` — calls POST `/api/auth/register`. `logout()` — clears `sessionStorage`, navigates to `/login`. `getUser()` — returns parsed user object from `sessionStorage`. `getToken()` — returns the JWT string. `updateProfile()` and `changePassword()` — call the matching backend endpoints.

SERVICE `app/services/ticket.service.ts`

Wraps all ticket API calls: `getTickets()`, `createTicket(req)`, `updateStatus(id, status)`, `rateTicket(id, rating)`, `searchTickets(query)`. All methods return Observables that the component subscribes to.

SERVICE `app/services/chat.service.ts`

CROSS-COMPONENT COMMUNICATION FOR THE CHATBOT

```
@Injectable({ providedIn: 'root' })
export class ChatService {
  private signal = new Subject<void>();
  open$ = this.signal.asObservable();
  open() { this.signal.next(); }
}
```

When "Live Support" is clicked in the sidebar, `chatService.open()` is called. The chatbot component has subscribed to `open$` in its `ngOnInit` — when the signal fires, the chatbot opens. This is the correct Angular pattern for sibling component communication without a parent-child relationship.

SERVICE

`app/services/analytics.service.ts` / `outage.service.ts` / `notification.service.ts` / `user.service.ts` / `toast.service.ts`

analytics.service.ts — Calls GET `/api/analytics/{role}` and returns the stats map.

outage.service.ts — Gets outages, creates outages, resolves outages, gets outages by city.

notification.service.ts — Gets unread notifications, marks them as read.

user.service.ts — CRUD for users (admin use): list all users, create employee, update, delete.

toast.service.ts — Shows temporary success/error messages at the bottom of the screen. Uses an RxJS BehaviorSubject so any component can trigger a toast.

Page Components

PAGE `pages/login/login.component.ts + .html`**THE ANIMATED LOGIN / SIGN UP LANDING PAGE**

active property (`'login' | 'signup' | null`) controls which panel is visible. When null, the landing page shows two clickable cards. When 'login', a CSS overlay animates in covering the full page with the login form. When 'signup', same but with the registration form.

The animation is pure CSS — no JavaScript animation library needed. The `.lp-form-overlay` class transitions from `opacity: 0; transform: translateX(60px)` to `opacity: 1; transform: translateX(0)` in 0.6s using `cubic-bezier`.

On successful login, reads `user.role` and navigates: ADMIN → /admin, MANAGER → /manager, REPRESENTATIVE → /representative, CUSTOMER → /customer.

The Indian city list is defined as `readonly LOCATIONS = ['Mumbai', 'Delhi', 'Bangalore', 'Hyderabad', 'Chennai', 'Kolkata', 'Pune', 'Jaipur']` for the signup location dropdown.

PAGE `pages/admin/admin.component.ts + .html`**ADMIN DASHBOARD — THE MOST FEATURE-RICH PAGE**

Tab system: Uses Angular Router query params (`?tab=dashboard`). `ngOnInit` subscribes to `route.queryParams` and calls the appropriate load function whenever the tab changes.

Dashboard tab: Calls analytics API, then uses `setTimeout(..., 100)` before drawing Chart.js charts to ensure the canvas elements have rendered in the DOM after the `@if` block shows them.

Employees tab: Full CRUD — list all employees, add new (with role/manager assignment), edit, delete. Uses a modal dialog (controlled by `showModal` boolean) for add/edit forms.

Tickets tab: Shows all tickets across all customers and reps. Admin can update any ticket's status.

Outages tab: Shows all outages. Admin can report a new outage (selecting city, domain, severity) and resolve existing ones.

PAGE `pages/representative/representative.component.ts + .html`**REP DASHBOARD — 3 TABS: DASHBOARD, MY TICKETS, SEARCH**

Dashboard tab: Shows stats cards (Open, In Progress, Closed tickets, Avg response time). Caches stats in `cachedStats` to avoid re-fetching when returning to the tab.

My Tickets tab: Lists all tickets assigned to this rep. A modal lets them change the status. The status badge changes colour automatically based on status (green=CLOSED, orange=IN_PROGRESS, red=OPEN).

Search tab: An input field with a `subject.pipe(debounceTime(400))` so the API is called only after the user stops typing for 400ms. Searches across customer name, phone, and city using the `/api/tickets/search?q=` endpoint.

PAGE `pages/customer/customer.component.ts + .html`

CUSTOMER DASHBOARD — 2 TABS: DASHBOARD, MY TICKETS

Dashboard tab: Shows stats (Open, In Progress, Closed). Checks for active outages in the customer's city — if any, shows an orange alert banner at the top with the affected areas. Shows current plans (Fiber, Mobile, DTH). Shows an "About NexCare" section mentioning all Indian cities.

My Tickets tab: Shows the customer's own tickets. They can raise a new ticket via a form modal (selecting domain, entering subject and description). Closed tickets show a star rating widget (1–5 stars) if not yet rated. All dates are formatted in Indian style using `en-IN` locale.

PAGE `pages/profile/profile.component.ts + .html`

Accessible by all roles from the sidebar "Profile" link. Shows two forms side by side: (1) Update Profile — name, phone, location. (2) Change Password — current password, new password, confirm new password. Both call the respective AuthController endpoints and show a toast on success or failure.

Shared Components

SHARED `shared/sidebar/sidebar.component.ts + .html`

THE LEFT NAVIGATION BAR — SHARED ACROSS ALL DASHBOARDS

Reads the logged-in user from `authService.getUser()` and uses the role to show different navigation items for each user type. Navigation is handled by a unified `queryParamsRoutes` map — each route string maps to a path and tab:

```
rep-tickets → { path: '/representative', tab: 'tickets' }  
customer-tickets → { path: '/customer', tab: 'tickets' }  
admin-emp → { path: '/admin', tab: 'employees' }
```

Special case: `customer-support` calls `this.chat.open()` to trigger the chatbot instead of navigating. The `activePage` input determines which item appears highlighted.

SHARED `shared/chatbot/chatbot.component.ts + .html`

THE FLOATING CHAT SUPPORT WIDGET

A floating  button appears in the bottom-right corner of all dashboards. Clicking it opens a chat panel. The chatbot has 12 keyword-response pairs (e.g., "bill" → "You can view your bill under the Plans section..."). Messages not matching any keyword get a default response suggesting the user raise a support ticket.

In `ngOnInit`, it subscribes to `chatService.open$` — so when the sidebar's "Live Support" link is clicked, the chatbot opens automatically. Auto-scrolls to the latest message using `@ViewChild` and `chatBody.nativeElement.scrollTop`.

SHARED `shared/topbar/topbar.component.ts + .html`

The top navigation bar on all dashboards. Shows the page title, a notification bell (with unread count badge), and the user's name. Clicking the bell fetches and shows notifications from `/api/notifications`. Clicking a notification marks it as read. The user name links to the profile page.

SHARED `shared/toast/toast.component.ts`

A temporary pop-up notification that slides in from the bottom. Other components call `toastService.show("message", "success")` or `toastService.show("error", "error")`. The toast automatically disappears after 3 seconds using `setTimeout`.

Global Styles

CSS `frontend/src/styles.css`

THE SINGLE STYLE SHEET FOR THE ENTIRE APPLICATION (~480 LINES)

CSS Variables (Lilac Theme):

```
:root {
  --primary: #8b5cf6;           /* main lilac purple */
  --primary-dark: #6d28d9;      /* darker shade for hover */
  --accent: #c4b5fd;           /* lighter accent */
  --bg: #f5f3ff;               /* page background */
  --surface: #ffffff;           /* card backgrounds */
}
```

Login Page Animation: The `.lp-form-overlay` starts with `opacity: 0; transform: translateX(60px)` and transitions to `opacity: 1; transform: translateX(0)` when the `.lp-`

`form-show` class is added. The easing function `cubic-bezier(.77,0,.18,1)` creates a smooth, natural-feeling animation.

Card hover effect: Dashboard cards use `transform: translateY(-4px)` on hover with `cubic-bezier(.34,1.56,.64,1)` — the "overshoot" easing gives a spring-like feel.

Responsive breakpoints: On screens narrower than 768px, the sidebar collapses, cards stack vertically, and the login page shows a single-column layout.

06

Database Tables and Columns

NexCare uses an H2 file database. The database file is stored at `./data/csddb.mv.db` relative to where the backend runs. Spring creates and updates all tables automatically based on the Java model classes — no manual SQL needed.

USERS table

Column	Type	Constraint	Purpose
ID	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
NAME	VARCHAR(255)	NOT NULL	Full name
EMAIL	VARCHAR(255)	UNIQUE, NOT NULL	Login username
PASSWORD	VARCHAR(255)	NOT NULL	BCrypt hash
ROLE	VARCHAR(50)	NOT NULL	ADMIN/MANAGER/REPRESENTATIVE/CUSTOMER
PHONE	VARCHAR(255)	—	Contact number
LOCATION	VARCHAR(255)	—	City
MANAGER_ID	BIGINT	—	For reps: references their manager's ID
FIRST_LOGIN	BOOLEAN	—	True if initial password not changed
CREATED_AT	TIMESTAMP	—	Account creation time

TICKETS table

Column	Type	Purpose
ID	BIGINT PK	Ticket number

CUSTOMER_ID, CUSTOMER_NAME, CUSTOMER_PHONE, CUSTOMER_LOCATION	BIGINT/VARCHAR	Who raised it (denormalized for history)
DOMAIN	VARCHAR	Internet Services, Mobile, DTH, etc.
SUBJECT	VARCHAR	Ticket title
DESCRIPTION	VARCHAR(2000)	Full details
STATUS	VARCHAR	OPEN / IN_PROGRESS / CLOSED
REP_ID, REP_NAME, MANAGER_ID	BIGINT/VARCHAR	Who is handling it
CREATED_AT, FIRST_RESPONSE_AT, RESOLVED_AT	TIMESTAMP	Timeline tracking
RESPONSE_TIME_HRS, RESOLUTION_TIME_HRS	DOUBLE	Calculated performance metrics
RATING	INTEGER	1–5 star customer rating

OUTAGES and NOTIFICATIONS tables

Table	Key columns
OUTAGES	id, location (city), affected_areas, domain, severity (HIGH/MEDIUM/LOW), status (ACTIVE/RESOLVED), description, reported_at, resolved_at
NOTIFICATIONS	id, user_id, message, type (TICKET/STATUS/OUTAGE), is_read (boolean), created_at

07 API Endpoints Reference

All endpoints are prefixed with `/api`. All except `/auth/login` and `/auth/register` require a valid JWT token in the Authorization header. The Angular JWT Interceptor adds this automatically.

Method	Endpoint	Who can call	What it does
POST	<code>/api/auth/login</code>	Anyone	Login — returns token + user
POST	<code>/api/auth/register</code>	Anyone	Sign up — creates CUSTOMER account
GET	<code>/api/auth/me</code>	All roles	Returns current user's profile
PUT	<code>/api/auth/profile</code>	All roles	Update name, phone, location
PUT	<code>/api/auth/change-password</code>	All roles	Change password
GET	<code>/api/tickets</code>	All roles	Get tickets (filtered by role)
POST	<code>/api/tickets</code>	CUSTOMER	Raise a new support ticket
PUT	<code>/api/tickets/{id}/status</code>	REP, MGR, ADMIN	Update ticket status
PUT	<code>/api/tickets/{id}/rate</code>	CUSTOMER	Rate a closed ticket
GET	<code>/api/tickets/search?q=</code>	REP, MGR, ADMIN	Search by customer name/phone/city
GET	<code>/api/analytics/admin</code>	ADMIN	Admin dashboard stats
GET	<code>/api/analytics/manager</code>	MANAGER	Manager/team stats
GET	<code>/api/analytics/rep</code>	REPRESENTATIVE	Rep's own performance stats
GET	<code>/api/analytics/customer</code>	CUSTOMER	Customer's ticket stats
GET	<code>/api/users</code>	ADMIN	List all employees
POST	<code>/api/users</code>	ADMIN	Create new employee
PUT	<code>/api/users/{id}</code>	ADMIN	Update employee details

DELETE	/api/users/{id}	ADMIN	Delete employee
GET	/api/outages	ADMIN, MANAGER	List all outages
POST	/api/outages	ADMIN	Report a new outage
PUT	/api/outages/{id}/resolve	ADMIN, MANAGER	Mark outage as resolved
GET	/api/outages/location?city=	CUSTOMER	Active outages in a city
GET	/api/notifications	All roles	Unread notifications for logged-in user
PUT	/api/notifications/{id}/read	All roles	Mark notification as read

08 Key Flows — Step by Step

Flow 1: Customer Raises a Support Ticket

1

Customer clicks "My Tickets" tab → clicks "Raise Ticket" button

Angular router navigates to `?tab=tickets`. Customer fills in domain (e.g., Internet Services), subject, and description.

2

Angular calls POST /api/tickets

JWT Interceptor adds the token. Spring Boot receives the request. JwtAuthFilter validates the token and confirms the caller is a CUSTOMER.

3

TicketController → TicketService.createTicket()

Builds a Ticket object with status=OPEN, customerId from the token, and all form fields. Saves to database.

4

Notifications sent to all reps

Service fetches all REPRESENTATIVE users and creates a notification for each: "New ticket #7 raised by Priya Sharma — Internet Services".

5

Response returned to Angular

The saved ticket is returned as JSON. Angular adds it to the tickets list. A toast notification shows "Ticket raised successfully".

Flow 2: Rep Updates Ticket Status to IN_PROGRESS

1

Rep clicks on an OPEN ticket → selects "In Progress" → submits

Angular calls PUT /api/tickets/{id}/status with body `{ "status": "IN_PROGRESS" }`

2

TicketService.updateStatus() runs

Checks that this is the first response (firstResponseAt == null). Records: rep's name and ID, current timestamp as firstResponseAt. Calculates responseTimeHrs = (now - createdAt) in hours.

3

Notification sent to customer

"Your ticket #3 status updated to IN PROGRESS" — saved to NOTIFICATIONS table.

4

Customer sees update

Next time the customer loads their tickets (or refreshes), the ticket shows "In Progress". The bell icon in their topbar shows a new notification count.

Flow 3: Admin Reports a Service Outage

1

Admin fills the outage form: city=Mumbai, domain=Internet Services, severity=HIGH

Angular calls POST /api/outages with the form data.

2

OutageController → OutageService saves the outage

Status is set to ACTIVE. reportedAt is recorded.

3

Mumbai customers see an alert banner on next dashboard load

Customer dashboard calls GET /api/outages/location?city=Mumbai. Returns the active outage. Angular displays: "Service Disruption — Internet Services disruption in Andheri, Bandra, Colaba. Our team is working to restore service."

4

Admin resolves the outage

PUT /api/outages/{id}/resolve. Status becomes RESOLVED, resolvedAt recorded. Customers in Mumbai no longer see the banner.

09

Security Architecture

Why JWT (JSON Web Tokens)?

Traditional session-based auth stores login state on the server. With JWT, the server is completely stateless — all user information is encoded inside the token itself. The server just needs to verify the signature. This scales better and fits REST API design perfectly.

What's inside a JWT token?

A JWT has three parts separated by dots: `header.payload.signature`

```
// Decoded payload of a NexCare JWT token:
{
  "sub": "amit.kumar@nexcare.in",    ← user's email (subject)
  "userId": 5,                      ← user's database ID
  "role": "REPRESENTATIVE",         ← their role
  "name": "Amit Kumar",              ← display name
  "managerId": 3,                   ← which manager they report to
  "iat": 1718000000,                 ← issued at (Unix timestamp)
  "exp": 1718086400                 ← expires after 24 hours
}
```

The token is signed with a secret key using HMAC-SHA256. Any tampering with the payload would invalidate the signature — the server detects this immediately and returns 401.

Dual Security — Frontend and Backend

Layer	Where	What it does	Can be bypassed?
Auth Guard	Angular (frontend)	Prevents loading protected pages without a valid session	Yes — by a technical user. That's why the backend layer exists.
JWT Filter	Spring Boot (backend)	Validates token signature on every API call. No valid token = 401 Unauthorized.	No — the backend is the final authority.

@PreAuthorize	Spring Boot (per-method)	Checks role even after JWT validation. Blocks cross-role API calls.	No.
---------------	-----------------------------	---	-----

Why two layers? The auth guard is for user experience — it prevents the UI from even trying to show an admin page to a customer. The JWT filter is for real security — even if someone uses Postman or a custom script to call the API, they cannot get any data without a valid token for the correct role.

10 Configuration Files

CONFIG frontend/proxy.conf.json

```
{
  "/api": {
    "target": "http://localhost:8080",
    "secure": false,
    "changeOrigin": true
  }
}
```

Angular's development server runs on port 4200. Spring Boot runs on port 8080. Browsers block requests from one port to another (CORS policy). This proxy configuration solves that — any request from Angular that starts with `/api` is silently forwarded to `http://localhost:8080`. To the browser, everything looks like it's on port 4200.

CONFIG frontend/angular.json

The main Angular build configuration. Key settings: `proxyConfig: "proxy.conf.json"` registers the proxy, `outputPath: "dist/frontend"` sets where the built files go, `budgets` warns if the bundle gets too large. The `styles` array points to `src/styles.css` as the global stylesheet.

CONFIG frontend/package.json

Lists all JavaScript/TypeScript packages the frontend needs. Key dependencies: `@angular/core`, `@angular/router`, `@angular/forms`, `@angular/common/http` (all @17.x), `chart.js` for the charts, `rxjs` for reactive programming. Running `npm install` reads this file and downloads everything listed.

CONFIG pom.xml (backend)

Maven's equivalent of package.json. Lists all Java dependencies: `spring-boot-starter-web` (REST API), `spring-boot-starter-security` (security), `spring-boot-starter-data-jpa` (database), `h2`

(H2 database driver), `jjwt-api/impl/jackson` 0.12.3 (JWT library), `lombok` (code generation), `resilience4j-spring-boot3` (circuit breaker). Maven downloads all of these from Maven Central on first run.

CONFIG `.vscode/tasks.json`

Defines run tasks for VS Code. The main task "Run NexCare (Full Stack)" is a compound task that starts both "Backend: Spring Boot" (`mvn spring-boot:run`) and "Frontend: Angular" (`npm install && npx ng serve --port 4200 --open`) simultaneously. These can be triggered from the VS Code menu: Terminal → Run Task.

NexCare — Interactive Dashboard System for Customer Service Operations

Rishitha Manyam | Wipro RGA Program 2025 | Java Full Stack Capstone Project